



Singapore University of Technology and Design

50.043 Database Systems

Group 2

SimpleDB - Lab1

<i>Authors</i>	<i>Student ID</i>
Jone Chong Jin	1006338
Sarang Nambiar	1006181
Loh Jianyang John	1006360

04/03/2025

Implementation Details:

1. Explain any design decisions you made.

Most of the design decisions of the implementation were made by following the instructions provided for the skeleton methods. However, certain design decisions were made with specific intentions in mind:

- a. `ConcurrentHashMap`: A thread-safe implementation of the `Map` interface in Java which allows multiple simultaneous read and write operations without external synchronization. This would allow the `Catalog` to easily scale for multiple read and write requests without causing race conditions. This `HashMap` has been implemented in `/simplifiedb/common/Catalog.java` and `/simplifiedb/storage/BufferPool.java`.
- b. `Objects.hash`: Inspired by the `hashCode` method implementation in `/simplifiedb/index/BTreePageId.java`, we decided to follow a similar implementation for the `hashCode` methods in `/simplifiedb/storage/HeapPageId.java` and `/simplifiedb/storage/RecordId.java`. We also acknowledge the existence of the built-in `Objects.hashCode()` method but decided to choose `Objects.hash()` since it provides us the flexibility to hash multiple attributes at once.
- c. `RandomAccessFile`: Provides random access to a file to read and write pages at arbitrary offsets. This Object and its methods were used in `/simplifiedb/storage/HeapFile.java`.

2. Explain the non-trivial part of your code.

`HeapFileIterator`: A custom class designed to efficiently iterate through all tuples in a `HeapFile`. Instead of loading all tuples into memory at once which could lead to out-of-memory errors for very large tables, this iterator lazily loads pages into the `BufferPool` one at a time. It then reads the tuples from each loaded page, ensuring memory efficiency. This makes `HeapFileIterator` a crucial and non-trivial component of our code.

3. Discuss and justify any changes you made to the API.

No changes were made to the original API for the implementation of this task.

4. Describe any missing or incomplete elements of your code.

In `/simplifiedb/storage/BufferPool.java`, our current implementation of the `getPage` method does not account for `Page Permissions` during the retrieval process. In future labs, we plan to implement a custom class which would

contain the Permissions Enum and the Page Object as its attributes and assign this custom class as the value for the key-value mapping for the original hashmap we created (PageId -> Page mapping). Alternatively, we could create a new ConcurrentHashMap with the mapping - Page/PageId -> Permissions to achieve similar results. Moreover, the getPage method also implements a simple locking mechanism but it may or may not align with future lab requirements, since specific implementation details are not mentioned.

Findings:

While trying out the final lab1 end-end test case with the `some_data_file.txt`, we encountered an interesting issue. The expected output from the test was supposed to be the printed field values for every record present in the text file. However, only the first two of three records were being outputted from the test. We got right to exploring what the possible cause could be and John found out that the code for parsing the data in the text file provided in `HeapFileEncoder` had a small issue. The convert function in the `HeapFileEncoder` counted the number of valid records based on the presence of a new line character (`\n`) which was not present in the text-snippet provided for `some_data_file.txt` in the GitHub. So, when we pasted the data from this snippet over to the text file, the convert function read the number of rows (recordcount) as 2.

```
if (recordcount >= nrecords
    || done && recordcount > 0
    || done && npages == 0) {
    int i = 0;
    byte headerbyte = 0;

    for (i=0; i<nheaderbits; i++) {
        if (i < recordcount)
            headerbyte |= (1 << (i % 8));

        if (((i+1) % 8) == 0) {
            headerStream.writeByte(headerbyte);
            headerbyte = 0;
        }
    }
}
```

Code snippet of convert function in HeapFileEncoder.java

The consequence of this was that the header byte for the last record was considered invalid and written over to the binary file version of the text file. Hence, the reason for the test case outputting only 2 of the 3 records provided in the text file.